



More Markets Vaults Core

Security Review



MORE

Disclaimer

Security Review

More Markets Vaults Core



Disclaimer

The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

Table of Contents

Security Review

More Markets Vaults Core



Table of Contents

Disclaimer	3
Summary	7
Scope	9
Methodology	9
Project Dashboard	13
Risk Section	16
Findings	18
3S-Vaults-H01	18
3S-Vaults-H02	21
3S-Vaults-H03	24
3S-Vaults-H04	26
3S-Vaults-H05	28
3S-Vaults-H06	30
3S-Vaults-H07	32
3S-Vaults-H08	34
3S-Vaults-H09	36
3S-Vaults-H10	38
3S-Vaults-M01	40
3S-Vaults-M02	42
3S-Vaults-M03	44
3S-Vaults-M04	46
3S-Vaults-M05	48
3S-Vaults-M06	50
3S-Vaults-M07	52
3S-Vaults-M08	54
3S-Vaults-M09	55
3S-Vaults-M10	57
3S-Vaults-M11	58
3S-Vaults-M12	60
3S-Vaults-M13	61
3S-Vaults-M14	62
3S-Vaults-L01	63
3S-Vaults-L02	64
3S-Vaults-L03	66
3S-Vaults-L04	67

Summary

Security Review

More Markets Vaults Core



Summary

Three Sigma audited More Markets in a 10 person week engagement. The audit was conducted from 29/09/2025 to 03/11/2025.

Protocol Description

MORE Vaults is an open-source cross-chain vault standard that combines the composability of the ERC-2535 Diamond Standard with a rigorously audited, invariant and generic core. By separating the asset-critical logic from the ever-evolving strategy layer, MORE drives down implementation and security costs for strategists, integrated protocols, and depositors alike.

Scope Security Review

More Markets Vaults Core



Scope

Filepath	nSLOC
src/libraries/MoreVaultsLib.sol	806
src/facets/VaultFacet.sol	746
src/facets/ERC4626Facet.sol	339
src/registry/VaultsRegistry.sol	166
src/facets/ERC7540Facet.sol	164
src/facets/MulticallFacet.sol	159
src/factory/VaultsFactory.sol	150
src/facets/ConfigurationFacet.sol	139
src/libraries/AccessControlLib.sol	138
src/facets/AccessControlFacet.sol	104
src/registry/BaseVaultsRegistry.sol	98
src/registry/OracleRegistry.sol	91
src/facets/DiamondLoupeFacet.sol	84
src/MoreVaultsDiamond.sol	76
src/facets/BaseFacetInitializer.sol	57
src/facets/DiamondCutFacet.sol	37
crosschain	1000
Total	4354

Methodology

Security Review

More Markets Vaults Core



To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

Taxonomy

In this audit, we classify findings based on ImmuneFi's [Vulnerability Severity Classification System \(v2.3\)](#) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

Impact / Likelihood	LOW	MEDIUM	HIGH
NONE	None		
LOW	Low		
MEDIUM	Low	Medium	Medium
HIGH	Medium	High	High
CRITICAL	High	Critical	Critical

Project Dashboard

Security Review

More Markets Vaults Core



Project Dashboard

Application Summary

Name	More Markets
Repository	https://github.com/MORE-Vaults/MORE-Vaults-Core
Commit	8f9e23f
Language	Solidity
Platform	Flow

Engagement Summary

Timeline	29/09/2025 to 03/11/2025
Nº of Auditors	2
Review Time	10 person weeks

Vulnerability Summary

Issue Classification	Found	Addressed	Acknowledged
Critical	0	0	0
High	10	10	0
Medium	14	14	0
Low	4	2	2
None	0	0	0

Category Breakdown

Suggestion	0
Documentation	0
Bug	28
Optimization	0
Good Code Practices	0

Risk Section Security Review

More Markets Vaults Core



Risk Section

No risks were identified.

Findings

Security Review

More Markets Vaults Core



Findings

3S-Vaults-H01

Disabling oracle accounting will be impossible when oracles fails due to checks in `removeFacetsForaccounting` function

Id	3S-Vaults-H01
Classification	High
Impact	High
Likelihood	Medium
Category	Bug
Status	Addressed in #cf4c602 .

Summary

Disabling oracle-based cross-chain accounting can become impossible if the total value of assets on remote spoke chains exceeds a small threshold (10e4). This happens because a balance check prevents facets that hold funds from being removed, as their removal could cause accounting issues. However, the `BridgeFacet` reports the funds held in spoke vaults, and an admin should still be able to remove it even when the reported value exceeds 10e4 while switching to LZ based cross-chain accounting.

Description

The `setOraclesCrossChainAccounting(false)` function is used to turn off oracle-based accounting for cross-chain vaults. Internally, it calls `MoreVaultsLib.removeFromFacetsForAccounting` to remove the `accountingBridgeFacet` selector from the vault's accounting facets list.

The `removeFromFacetsForAccounting` function contains a critical safety check to prevent removing a facet that is still managing funds. It performs a `staticcall` to the facet's accounting function and reverts if the returned value is greater than **10e4**.

```
// In MoreVaultsLib.removeFromFacetsForAccounting
(bool success, bytes memory result) =
address(this).staticcall(abi.encodeWithSelector(selector));
// ...
```

```

uint256 decodedAmount = abi.decode(result, (uint256));
if (decodedAmount > 10e4) {
    revert FacetHasBalance(decodedAmount);
}

```

While this check is logical for facets that directly hold funds (like ERC4626Facet), it is incorrectly applied to BridgeFacet too. The **accountingBridgeFacet** function returns the total value of assets held across all remote spoke chains, not a balance held on the current chain.

A more critical DoS scenario arises if an oracle for any spoke chain fails (for example, due to returning zero or stale data). In such cases, the admin must disable oracle-based cross-chain accounting to restore normal operation. However, this operation requires calling **accountingBridgeFacet** from within **removeFromFacetsForAccounting**.

Since **accountingBridgeFacet** calls **getSpokeValue** for every spoke, a single failing oracle will cause it to revert, which then causes **setOraclesCrossChainAccounting(false)** to revert. As a result, the vault becomes stuck in a state where the failing oracle makes it impossible to switch to the alternative LZ-based cross-chain accounting method.

Recommendation

The root cause of both the failing oracle DoS and the incorrect balance check is the unconditional staticcall within the **removeFromFacetsForAccounting** function. This check is necessary for facets that hold local funds (like ERC4626Facet) but is inappropriate and dangerous for the BridgeFacet, which only reports on remote funds.

The recommended solution is to modify **removeFromFacetsForAccounting** to bypass the staticcall and the subsequent balance check specifically when the facet being removed is the **accountingBridgeFacet**.

This single change fixes both problems:

1. Failing Oracle: By not calling **accountingBridgeFacet**, a failing spoke oracle can no longer cause a revert.
2. Balance Check: With no call being made, there is no balance to check, so the incorrect **FacetHasBalance** revert is avoided.

This approach allows admin to disable oracle accounting when needed, without affecting the safety checks for other accounting facets.

```

// In MoreVaultsLib.sol
function removeFromFacetsForAccounting(MoreVaultsStorage storage ds, bytes4 selector,
bool _isReplacing) internal {
    for (uint256 i; i < ds.facetsForAccounting.length;) {
        if (ds.facetsForAccounting[i] == selector) {
            if (!_isReplacing) {
                // --- PROPOSED CHANGE ---
                // @audit Only perform the balance check for facets other than the bridge
                accounting facet.
                bytes4 accountingBridgeFacetSelector =
                bytes4(keccak256(abi.encodePacked("accountingBridgeFacet()")));
                if (selector != accountingBridgeFacetSelector) {
                    (bool success, bytes memory result) =
                address(this).staticcall(abi.encodeWithSelector(selector));
                    if (!success) {
                        revert AccountingFailed(selector);
                    }
                    uint256 decodedAmount = abi.decode(result, (uint256));
                    if (decodedAmount > 10e4) {
                        revert FacetHasBalance(decodedAmount);
                    }
                }
            }
            ds.facetsForAccounting[i] = ds.facetsForAccounting[ds.facetsForAccounting.length -
1];
            ds.facetsForAccounting.pop();
            break;
        }
        unchecked {
            ++i;
        }
    }
}

```

3S-Vaults-H02

Oracle value not converted to underlying asset in
accountingBridgeFacet

Id	3S-Vaults-H02
Classification	High
Impact	High
Likelihood	High
Category	Bug
Status	Addressed in #17fb43b .

Summary

The **accountingBridgeFacet()** function in **BridgeFacet.sol** incorrectly returns a raw USD value from the oracle without converting it to the vault's underlying asset, unlike **accountingERC4626Facet()** and **accountingERC7540Facet()**, which perform this conversion.

Description

The **totalAssets()** function in **VaultFacet.sol** relies on a helper function, **_accountFacets**, to aggregate values from all of the accounting facets. This **_accountFacets** function operates under the assumption that all accounting facets return values denominated in the vault's underlying asset. It directly sums up the amounts returned by these facets.

Other accounting facets, such as **accountingERC4626Facet** and **accountingERC7540Facet**, use **MoreVaultsLib.convertToUnderlying** function to convert the assets value into the vault's underlying asset before returning it. But **accountingBridgeFacet()** function directly returns the sum of values returned by the oracles. As the accounting oracles return the values of **totalAssetsUsd()** function on spoke chain vaults, it will be in USD denomination and needs to be converted to vaults underlying asset.

```
function accountingBridgeFacet() public view returns (uint256 sum, bool isPositive) {
    MoreVaultsLib.MoreVaultsStorage storage ds = MoreVaultsLib.moreVaultsStorage();
    IVaultsFactory factory = IVaultsFactory(ds.factory);
    uint32 localEid = factory.localEid();
```

```

    (uint32[] memory eids, address[] memory vaults) = factory.hubToSpokes(localEid,
address(this));
    for (uint256 i = 0; i < vaults.length;) {
        IMoreVaultsRegistry registry =
IMoreVaultsRegistry(AccessControlLib.vaultRegistry());
        IOracleRegistry oracle = registry.oracle();
        sum += oracle.getSpokeValue(address(this), eids[i]); // @audit
        unchecked {
            ++i;
        }
    }
    return (sum, true);
}

```

Recommendation

Convert the **sum** value to underlying amount in **accountingBridgeFacet** using **MoreVaultsLib.convertToUnderlying** function.

```

function accountingBridgeFacet() public view returns (uint256 sum, bool isPositive) {
    MoreVaultsLib.MoreVaultsStorage storage ds = MoreVaultsLib.moreVaultsStorage();
    IVaultsFactory factory = IVaultsFactory(ds.factory);
    uint32 localEid = factory.localEid();
    (uint32[] memory eids, address[] memory vaults) = factory.hubToSpokes(localEid,
address(this));
    for (uint256 i = 0; i < vaults.length;) {
        IMoreVaultsRegistry registry =
IMoreVaultsRegistry(AccessControlLib.vaultRegistry());
        IOracleRegistry oracle = registry.oracle();
        sum += oracle.getSpokeValue(address(this), eids[i]);
        unchecked {
            ++i;
        }
    }
+   sum = MoreVaultsLib.convertUsdToUnderlying(sum, Math.Rounding.Floor);
    return (sum, true);
}

```

3S-Vaults-H03

Malicious owner can drain vault by linking fake/unrelated spoke vaults

Id	3S-Vaults-H03
Classification	High
Impact	High
Likelihood	Medium
Category	Bug
Status	Addressed in #31845ea .

Summary

A malicious vault owner can link an unrelated spoke vault to a hub vault, as long as they own both. This allows them to artificially inflate the hub's share price by manipulating cross-chain accounting, enabling the owner to withdraw their shares at a higher value and potentially drain the vault of legitimate user assets.

Description

The **registerSpoke** function in **VaultsFactory.sol** checks that the owner of a spoke vault matches the owner of the hub vault before linking them. However, this check is insufficient. A malicious owner can link any spoke vault they control, even if it is completely unrelated to the hub vault.

If an owner links a spoke vault to an unrelated hub vault, the hub vault's accounting can become corrupted, as assets from unrelated meshes will be incorrectly included in calculations, inflating the share price. A malicious owner can exploit this by redeeming their shares at the inflated value, effectively stealing funds from other vault users.

Attack steps:

Setup:

- Attacker (Alice) controls two independent vault meshes:
 - mesh_A: hub_vault_a (Ethereum), spoke_vault_a (Arbitrum)
 - mesh_B: hub_vault_b (Ethereum), spoke_vault_b (Arbitrum)

- registerSpoke only verifies that spoke.owner == hub.owner and does not check that the spoke belongs to the same mesh.

Steps:

1. Alice, as owner of hub_vault_a and spoke_vault_b, calls registerSpoke(hub_vault_a, spoke_vault_b).
2. The factory accepts the registration because ownership matches, even though spoke_vault_b is from a different mesh.
3. Accounting and asset data from spoke_vault_b (Arbitrum) are now (incorrectly) included when computing hub_vault_a's (Ethereum) total assets / share price.
4. The share price of hub_vault_a becomes inflated by the unrelated spoke's assets.
5. Alice has some shares in the hub_vault_a and redeems them while the share price is inflated.
6. Alice withdraws more value than her fair share, the excess is taken from legitimate hub_vault_a depositors.

Recommendation

When adding a spoke, verify whether a spoke from the same chain is already linked to the hub vault. If a spoke from that chain already exists, prevent linking an additional spoke from the same chain.

3S-Vaults-H04

stale totalAssets usage during cross-Chain finalization leads to theft of funds

Id	3S-Vaults-H04
Classification	High
Impact	High
Likelihood	High
Category	Bug
Status	Addressed in #c5da9dd .

Summary

The vault's **totalAssets** is cached when a cross-chain action is initiated, and this stale value is later used during finalization. If totalAssets changes between initiation and finalization due to other requests, it can lead to incorrect share calculations and potential value extraction from the vault.

Description

The **_createCrossChainRequest** function is called to initiate an asynchronous cross-chain action, such as a deposit or withdrawal. During this process, it calculates and stores the vault's current **totalAssets** in the `guidToCrossChainRequestInfo` struct:

```
MoreVaultsLib.CrossChainRequestInfo memory requestInfo =
MoreVaultsLib.CrossChainRequestInfo({
    ...
    totalAssets: IVaultFacet(address(this)).totalAssets() // Caches point-in-time value
});
ds.guidToCrossChainRequestInfo[guid] = requestInfo;
```

This cached totalAssets value is then used during the finalization of the action to calculate the number of shares to mint or burn. The issue is that other request can execute between the initiation and finalization of current request which will alter the vault's actual **totalAssets** and totalSupply, in that case the current request will be finalized with outdated totalAssets.

This creates a race condition that can be exploited. An attacker can initiate a cross-chain deposit, and then, while that request is in-flight, another (potentially attacker-controlled) large deposit can be made to the vault. If the attacker's original deposit request is finalized after the second deposit request, the share calculation will use the stale, lower **totalAssets** value that was cached before the second deposit.

Attack steps

1. Let's say there are 100 assets and 100 shares in the vault.
2. A normal user tries to make a deposit request of 50 assets, but the attacker front-runs it by submitting a deposit request of 10 assets first.
3. In the attacker's request, **totalAssets** is stored as 100.
4. The normal user's deposit is then finalized, updating the vault state to **totalAssets** = 150 and **totalShares** = 150.
5. When the attacker's request is finalized, it still uses the stale **totalAssets** = 100, resulting in the attacker receiving 15 shares ($10 * 150 / 100$) instead of the correct 10 shares.
6. Now **totalAssets** = 160 but **totalShares** = 165, so the attacker can redeem 15 shares for 14.5 assets ($15 * 160 / 165$) even though he only deposited 10 assets, effectively stealing 4.5 assets from the protocol.

Recommendation

Do not cache the **totalAssets** value at the time of request initiation. The **totalAssets** field should be removed from the **CrossChainRequestInfo** struct.

The share calculation logic, performed during the finalization of the cross-chain request, must read the fresh, up-to-date **totalAssets()** value directly from the vault at the moment of execution. This ensures that the calculation always uses the most recent and accurate state of the vault, preventing this vulnerability.

3S-Vaults-H05

Permanent fund locking due to insufficient refund fees

Id	3S-Vaults-H05
Classification	High
Impact	High
Likelihood	Medium
Category	Bug
Status	Addressed in #5c8f918 .

Description

The **refundDeposit** function in **MoreVaultsComposer** uses a pre-stored **deposit.msgValue** to cover LayerZero fees for the cross-chain refund operation. If this value becomes insufficient due to fee volatility, the refund will fail and user funds will be permanently locked without any recovery mechanism.

The issue manifests in the following flow:

1. Deposit Initiation: When **_initDeposit** is called, the function stores **msg.value - readFee** as **deposit.msgValue** in the **pendingDeposits** mapping:

```
pendingDeposits[guid] = PendingDeposit(  
    _depositor,  
    _tokenAddress,  
    _oftAddress,  
    _assetAmount,  
    _refundAddress,  
    msg.value - readFee, // Fixed value stored  
    _srcEid,  
    _sendParam  
);
```

2. Fee Validation: The **minMsgValue** parameter is validated in **handleCompose** to ensure sufficient fees were provided at deposit initiation. However, this validation only checks that fees were sufficient **at that moment**, not for future operations.

3. Refund Execution: When **refundDeposit** is called, it uses the stored **deposit.msgValue** for the cross-chain send operation:

```
IOFT(deposit.tokenAddress).send{value: deposit.msgValue}(
    refundSendParam, MessagingFee(deposit.msgValue, 0), deposit.refundAddress
);
```

If the **IOFT.send** operation reverts due to insufficient fees (e.g., LayerZero fees increased between deposit initiation and refund), the entire transaction reverts.

Fund Locking Scenario:

1. User initiates async deposit with **msg.value = 0.1 ETH** (after accounting fees), which is sufficient for LayerZero at that moment
2. Deposit fails and needs to be refunded
3. Between deposit initiation and refund, LayerZero fees increase to 0.12 ETH due to network congestion
4. **refundDeposit** is called with the stored **deposit.msgValue = 0.1 ETH**
5. The **IOFT.send** operation reverts due to insufficient fees
6. User's deposited tokens remain in the contract with no recovery mechanism

The function is declared as **payable**, suggesting it could accept additional **msg.value**, but this value is never used - only the pre-stored **deposit.msgValue** is used for the refund send operation. There is no way to:

- Provide additional fees through **msg.value**
- Retry with updated fee values
- Recover the pending deposit through any other means

The only recovery option would be a contract upgrade, which may not be possible or timely.

Recommendation

Modify **refundDeposit** to accept additional **msg.value** to supplement the stored **deposit.msgValue**, ensuring sufficient fees are available even if LayerZero fees have increased:

3S-Vaults-H06

Token substitution attack in **MoreVaultsComposer**

Id	3S-Vaults-H06
Classification	High
Impact	High
Likelihood	High
Category	Bug
Status	Addressed in #a797f0f .

Description

The **MoreVaultsComposer** is vulnerable to a token substitution attack where an attacker can steal tokens held by the composer contract. The attack exploits two things:

- an exception in the **IzCompose** validation, allowing execution to continue even if **_composeSender** is not trusted if the underlying token for this OFT is not depositable.

```
function IzCompose(
    address _composeSender, // The OFT used on refund, also the vaultIn token.
    bytes32 _guid,
    bytes calldata _message, // expected to contain a composeMessage =
abi.encode(SendParam hopSendParam,uint256 minMsgValue)
    address, /*_executor*/
    bytes calldata /*_extraData*/
) external payable virtual override {
    if (msg.sender != ENDPOINT) revert OnlyEndpoint(msg.sender);
    if (!LzAdapter(VAULT_FACTORY.IzAdapter()).isTrustedOFT(_composeSender)) {
        if
(IConfigurationFacet(address(VAULT)).isAssetDepositable(IOFT(_composeSender).token()
    )) {
        revert InvalidComposeCaller(_composeSender);
    }
}
```

- the fact that the function **tokenis** is called twice during the deposit flow, allowing a malicious OFT adapter to return different addresses on each call.

Attack Scenario:

1. Setup:

- Attacker deploys a malicious OFT adapter for a worthless token
- The adapter implements state-dependent token behavior:
 - First call: returns the worthless token address (not depositable)
 - Subsequent calls: return a depositable token address (e.g., USDC)

2. Cross-chain Send:

- Attacker initiates a cross-chain OFT send from source chain to hub chain
- Sends amount of worthless tokens
- Specifies **MoreVaultsComposer** as the recipient with compose message

3. Security Check Bypass (**lzCompose**):

- LayerZero Endpoint calls **MoreVaultsComposer.lzCompose**
- Security check executes: **IOFT(_composeSender).token** (first call)
- Returns worthless token -> not depositable -> check passes
- Malicious OFT adapter is not trusted, but token is not depositable, so no revert occurs

4. Token Substitution (**handleCompose**):

- **handleCompose** calls **IOFT(_ofIn).token** again (second call)
- Now returns depositable token address (USDC)
- **_initDeposit** or **_depositAndSend** proceeds with USDC as the token
- Attacker controls **_amount** parameter via the compose message
- Vault deposit executes using USDC from Composer's balance, not attacker's funds

5. Result:

- Attacker receives vault shares for stolen tokens
- Legitimate user's pending deposit funds are stolen
- No actual transfer from attacker occurred

Recommendation

Prohibit the use of OFT tokens that are not trusted, even if their underlying tokens are not depositable.

3S-Vaults-H07

Missing locked token accounting in **ERC7540Facet**

Id	3S-Vaults-H07
Classification	High
Impact	High
Likelihood	High
Category	Bug
Status	Addressed in #bed0f7b .

Description

The **accountingERC7540Facet** function does not properly account for assets during pending asynchronous requests, leading to underreported vault balances.

```
function accountingERC7540Facet() public view returns (uint256 sum, bool isPositive) {
    MoreVaultsLib.MoreVaultsStorage storage ds = MoreVaultsLib.moreVaultsStorage();
    for (uint256 i = 0; i < ds.tokensHeld[ERC7540_ID].length(); i++) {
        address vault = ds.tokensHeld[ERC7540_ID].at(i);
        if (ds.isAssetAvailable[vault]) {
            unchecked {
                ++i;
            }
            continue;
        }
        address asset = IERC4626(vault).asset();
        uint256 balance = IERC20(vault).balanceOf(address(this));
        uint256 convertedToVaultUnderlying = IERC4626(vault).convertToAssets(balance);
        sum += MoreVaultsLib.convertToUnderlying(asset, convertedToVaultUnderlying,
Math.Rounding.Floor);
        unchecked {
            ++i;
        }
    }
    return (sum, true);
}
```


When a user initiates an asynchronous deposit via **erc7540RequestDeposit**, the underlying assets are immediately transferred to the external **ERC7540** vault, but shares are not received until the request is finalized. During this pending period, **accountingERC7540Facet** only counts the vault shares currently held in the contract's balance, completely ignoring the transferred assets that are locked in the pending request.

The same issue occurs during asynchronous redemptions - shares are transferred out but assets haven't been received yet, causing those shares to disappear from accounting.

This creates serious issues: underreported TVL, incorrect share pricing, broken cross-chain accounting. The protocol already solved this exact problem in **ERC4626Facet** by tracking locked tokens, but the same logic was not applied to **ERC7540Facet**.

Recommendation

Implement the same locked token tracking mechanism that already exists in **ERC4626Facet** to ensure accurate accounting during asynchronous operations.

3S-Vaults-H08

An attacker can steal the native value from the vault, due to lack of `msg.value` check in **finalizeRequest** function

Id	3S-Vaults-H08
Classification	High
Impact	High
Likelihood	High
Category	Bug
Status	Addressed in #0e27487 .

Summary

When processing a **MULTI_ASSETS_DEPOSIT** action, the **finalizeRequest()** function in **BridgeFacet** executes a payable self call using a value decoded directly from user-controlled calldata (`actionCallData`). It fails to verify that this amount was actually sent by the caller of `finalizeRequest`. This allows an attacker to force the contract to fund the deposit with its own native token balance, leading to a direct loss of funds.

Description

The **finalizeRequest** function is payable and serves to execute the final step of a cross-chain transaction. For a **MULTI_ASSETS_DEPOSIT** action, the function decodes the parameters for the deposit from **requestInfo.actionCallData**, which includes an array of tokens, amounts, a receiver address, and a uint256 value intended for native token deposits.

The issue lies in this line:

```
(success, result) = address(this).call{value: value}{
    abi.encodeWithSelector(...)
};
```

The function uses the decoded value to fund a deposit call. However, it never validates that the **msg.value** of the external **finalizeRequest** call is equal to this decoded value.

Attack Steps

1. Attacker observes diamond contract has 10 ETH balance
2. Attacker calls **initVaultActionRequest()** with:
 - * `actionType = MULTI_ASSETS_DEPOSIT`
 - * `actionCallData = encoded(tokens[], amounts[], attacker_address, 10 ETH)`
 - * This creates a request with GUID
3. When the action is finalized after the cross-chain lookup, the attacker calls **finalizeRequest(GUID)** with `msg.value = 0`
4. `finalizeRequest()` decodes 10 ETH value from stored calldata
5. Makes self-call to deposit with value: 10 ETH
6. 10 ETH is taken from contract's balance and credited to attacker's deposit

Recommendation

Add validation in **finalizeRequest()** to ensure `msg.value` matches the native token value encoded in `actionCallData` for **MULTI_ASSETS_DEPOSIT** action.

3S-Vaults-H09

Selector validation bypass in **submitActions** allows curators to execute unauthorized actions

Id	3S-Vaults-H09
Classification	High
Impact	High
Likelihood	High
Category	Bug
Status	Addressed in #b249aa1 .

Description

In **MulticallFacet.submitActions**, permission validation is performed only on the first selector in the **actionsData** array:

```
function submitActions(bytes[] calldata actionsData) external override returns (uint256
nonce) {
    if (actionsData.length == 0) revert EmptyActions();
    bytes4 selector = bytes4(actionsData[0][:4]); // ← Only first selector checked
    AccessControlLib.validatePermissionForSelector(msg.sender, selector);
    // ... stores ALL actionsData
    ds.pendingActions[nonce] = MoreVaultsLib.PendingActions({actionsData: actionsData,
pendingUntil: pendingUntil});
```

However, when the actions are executed in **executeActions**, all actions in the array are executed via **_multicall** without additional validation:

```
function executeActions(uint256 actionsNonce) public override nonReentrant {
    // ...
    _multicall(actions.actionsData); // ← Executes all actions
```

This allows a curator to bypass permission controls by submitting multiple actions where the first action uses an allowed selector, but subsequent actions contain selectors they are not authorized to execute. All actions will be executed successfully.

Recommendation

Validate permissions for all selectors in the **actionsData** array, not just the first one:

```
function submitActions(bytes[] calldata actionsData) external override returns (uint256
nonce) {
    if (actionsData.length == 0) revert EmptyActions();
    // Validate all selectors
    for (uint256 i = 0; i < actionsData.length; i++) {
        bytes4 selector = bytes4(actionsData[i][:4]);
        AccessControlLib.validatePermissionForSelector(msg.sender, selector);
    }
    // ... rest of the function
}
```

3S-Vaults-H10

Unit mismatch in **lockedTokens** updates during async finalize (shares vs assets)

Id	3S-Vaults-H10
Classification	High
Impact	High
Likelihood	High
Category	Bug
Status	Addressed in #f27f7c7 .

Description

In **ERC4626Facet.genericAsyncActionExecution**, asynchronous deposit/withdraw is implemented as a two-step flow (request → finalize). Balances for assets/shares therefore update in two parts. To account for this, a **lockedTokens** mapping is used to prevent double counting. For example for an async deposit, the sent assets are tracked in **lockedTokens** until the corresponding shares are actually minted, when that temporary lock should be cleared.

The problem is that when finalizing a deposit, the code subtracts shares from **lockedTokens[balances.asset]** which tracks assets, instead of subtracting the corresponding asset amount. Since the assets:shares ratio is not always 1:1, this produces an accounting error.

```
if (balances.totalSupplyAfter > balances.totalSupplyBefore) {
  ds.lockedTokens[balances.asset] -= balances.sharesAfter - balances.sharesBefore;
}
```

A similar issue appears on finalize for withdraw:

```
if (balances.totalSupplyBefore > balances.totalSupplyAfter) {
  ds.lockedTokens[vault] -= balances.assetsAfter - balances.assetsBefore;
  MoreVaultsLib.removeTokenIfnecessary(ds.tokensHeld[ERC4626_ID], vault);
}
```

Recommendation

Ensure that when clearing **lockedTokens** during finalization, you update it using the correct unit.

3S-Vaults-M01

Incorrect msg.sender check prevents withdrawals when withdrawalQueue and oracle accounting is disabled

Id	3S-Vaults-M01
Classification	Medium
Impact	Medium
Likelihood	Medium
Category	Bug
Status	Addressed in #0421a38 .

Summary

The **withdrawFromRequest** function in **MoreVaultsLib.sol** uses an incorrect check to validate the initiator of a withdrawal when the withdrawal queue is disabled. When the withdrawal request is initiated from **initVaultActionRequest** function the BridgeFacet calls the withdraw function on behalf of owner via **finalizeRequest** function, so msg.sender won't be share owner in this case and withdraw function will revert.

Description

The **withdraw** function in **VaultFacet.sol** is the entry point for users to withdraw assets. It calls the internal library function **MoreVaultsLib.withdrawFromRequest** to verify if the withdrawal can proceed.

When the withdrawal queue is disabled (**isWithdrawalQueueEnabled == false**), **withdrawFromRequest** is supposed to allow only share owner to make withdrawal request. The logic for this is:

```
// if withdrawal queue is disabled, request can be processed immediately
if (!ds.isWithdrawalQueueEnabled) {
    // only allow for the shares owner to withdraw in this case
    return msg.sender == _requester;
}
```

But when withdrawal request is initiated from BridgeFacet, msg.sender will be diamond address ,therefore, the check **msg.sender == _requester** will always evaluate to **false**,

incorrectly blocking the withdrawal. While the **__getInfoForAction** function correctly determines the true initiator of the transaction, this address is not passed down to or used by **withdrawFromRequest**.

Recommendation

To resolve this, the **withdrawFromRequest** function must be modified to use the correct address of the transaction initiator instead of relying on **msg.sender**.

1. **Update Function Signature:** Modify the **withdrawFromRequest** function in **MoreVaultsLib.sol** to accept the initiator's address as an argument.

```
- function withdrawFromRequest(address _requester, uint256 _shares) internal returns
(bool)
+ function withdrawFromRequest(address _msgSender, address _requester, uint256
_shares) internal returns (bool)
```

2. **Update Validation Logic:** Change the validation check to use the newly passed-in **_msgSender** parameter.

```
- return msg.sender == _requester;
+ return _msgSender == _requester;
```

3. **Update Call Site:** In **VaultFacet.sol**, update the call to **withdrawFromRequest** to pass the correct initiator address, which is retrieved from **__getInfoForAction**.

```
- bool isWithdrawable = MoreVaultsLib.withdrawFromRequest(owner, shares);
+ bool isWithdrawable = MoreVaultsLib.withdrawFromRequest(msgSender, owner,
shares);
```

This ensures that the validation correctly compares the actual user's address with the share owner's address, allowing withdrawals to proceed as expected when the queue is disabled.

3S-Vaults-M02

Cross-Chain deposits fail when oracle accounting is enabled

Id	3S-Vaults-M02
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Addressed in #6b0557e .

Summary

The internal **handleCompose** function incorrectly initiates an async cross-chain flow when the vault is a cross-chain vault, without checking whether oracle accounting is enabled. Due to this, all cross-chain deposits will revert when oracle accounting is enabled, as the **initVaultActionRequest** function in BridgeFacet reverts when oracle accounting is active.

Description

The **MoreVaultsComposer.handleCompose** function is responsible for routing incoming cross-chain deposit requests. It checks if the destination vault is a "cross-chain vault" and, if so, unconditionally routes the transaction to the **_initDeposit** function to handle it via an asynchronous flow:

```
// MoreVaultsComposer.sol
if (VAULT_FACTORY.isCrossChainVault(uint32(VAULT_EID), address(VAULT))) {
    _initDeposit(_composeFrom, IOFT(_ofIn).token(), _ofIn, _amount, sendParam, tx.origin,
_srcEid);
} else {
    _depositAndSend(...);
}
```

The flaw is that this routing decision is made without considering the vault's accounting mode. The **_initDeposit** function proceeds to call **initVaultActionRequest** on the vault's **BridgeFacet**.

However, the **initVaultActionRequest** function contains an explicit check that causes it to revert if the vault has oracle accounting enabled:

```
// BridgeFacet.sol
if (ds.oraclesCrossChainAccounting) {
    revert AccountingViaOracles();
}
```

Recommendation

Only initiate cross-chain deposit flow, when the vault is cross-chain vault and the oracle accounting mode is disabled.

3S-Vaults-M03

Missing slippage protection in **completeDeposit**

Id	3S-Vaults-M03
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Addressed in #9d9466e .

Description

The **completeDeposit** function in **MoreVaultsComposer** does not validate the **minAmountLD** parameter before completing an async deposit, exposing users to unlimited slippage between deposit initiation and completion.

When a user initiates an async deposit via **_initDeposit**, they provide a **SendParam** structure that includes **minAmountLD** - the minimum number of shares they expect to receive. This parameter is stored in **pendingDeposits[guid].sendParam.minAmountLD**. However, when **completeDeposit** is called to finalize the deposit, the function:

1. Executes the vault deposit and receives **shares**
2. Sets **deposit.sendParam.minAmountLD = 0** without any validation
3. Proceeds to send the shares to the destination

```
function completeDeposit(bytes32 _guid) external virtual nonReentrant {
    if (msg.sender != address(VAULT) && msg.sender !=
address(VAULT_FACTORY.lzAdapter())) {
        revert OnlyVaultOrLzAdapter(msg.sender);
    }
    PendingDeposit memory deposit = pendingDeposits[_guid];
    if (deposit.assetAmount == 0) revert DepositNotFound(_guid);
    uint256 shares = _deposit(_guid, deposit.tokenAddress);
    deposit.sendParam.amountLD = shares;
    deposit.sendParam.minAmountLD = 0;
}
```

This unconditionally zeroes out the slippage parameter without checking if the actual **shares** received meet the user's minimum requirement.

In contrast, the synchronous flow in `_depositAndSend` properly validates slippage:

```
shareAmount = VAULT.deposit(_assetAmount, address(this));  
_assertSlippage(shareAmount, _sendParam.minAmountLD);
```

This discrepancy means users are protected from slippage in synchronous deposits but completely unprotected in async deposits, even though async deposits have a higher risk of unfavorable price movements due to the time delay between initiation and completion.

The user has no recourse and must accept whatever share amount they receive, regardless of how unfavorable the conversion rate has become.

Recommendation

Add slippage validation before completing the deposit.

3S-Vaults-M04

Performance fee charged on price fluctuations without real profit

Id	3S-Vaults-M04
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Addressed in #083f139 .

Description

The current implementation of **VaultFacet** charges performance fees whenever **_totalAssets** > **lastTotalAssets**, regardless of whether this increase represents actual new profit or simply price fluctuations of the underlying assets.

```
function _accruedFeeShares(uint256 _totalAssets) internal view returns (uint256
feeShares) {
    MoreVaultsLib.MoreVaultsStorage storage ds = MoreVaultsLib.moreVaultsStorage();
    uint256 lastTotalAssets = ds.lastTotalAssets;
    uint256 totalInterest = _totalAssets > lastTotalAssets ? _totalAssets - lastTotalAssets : 0;
    ...
}
```

The problem occurs in the following scenario:

1. Initial state: **totalAssets = 1000 USD**, **lastTotalAssets = 1000 USD**
2. Market moves up: **totalAssets = 1100 USD** -> Fee is charged on 100 USD gain, **lastTotalAssets = 1100 USD**
3. Market drops: **totalAssets = 1050 USD** -> No fee charged (**totalAssets < lastTotalAssets**)
4. Market recovers: **totalAssets = 1080 USD** -> Fee is charged again on 80 USD "gain", **lastTotalAssets = 1080 USD**

In this example, the vault has actually lost value (from 1100 USD peak to 1080 USD), yet fees were charged on the recovery from 1050 USD to 1080 USD. The vault has not reached a new profit level, but users are still paying performance fees on price fluctuations.

This creates an unfair situation where:

- Users pay multiple fees on the same profit range during market volatility

- Performance fees are charged even when the vault hasn't achieved new highs
- The protocol benefits from asset price oscillations rather than actual performance
- Long-term returns are eroded by repeated fees on the same value ranges

3S-Vaults-M05

No recovery mechanism for LayerZero dust accumulation in **MoreVaultsComposer**

Id	3S-Vaults-M05
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Addressed in #1f7f976 .

Description

LayerZero's decimal normalization mechanism truncates token amounts during cross-chain transfers, leaving dust locked in **MoreVaultsComposer** without any way to recover it. The protocol also incorrectly reports full deposit amounts in events while users receive fewer tokens than expected.

When **MoreVaultOfAdapter** sends tokens cross-chain through LayerZero's OFTCore, amounts are normalized to **sharedDecimals** (6 decimals by default). For tokens with 18 decimals precision, this truncation removes the 12 least significant digits.

```
function _debitView(
    uint256 _amountLD,
    uint256 _minAmountLD,
    uint32 /*_dstEid*/
) internal view virtual returns (uint256 amountSentLD, uint256 amountReceivedLD) {
    // @dev Remove the dust so nothing is lost on the conversion between chains with
    // different decimals for the token.
    amountSentLD = _removeDust(_amountLD);
    // @dev The amount to send is the same as amount received in the default
    // implementation.
    amountReceivedLD = amountSentLD;
```

The truncation happens in LayerZero's internal logic, which rounds down to the nearest **sharedDecimals** unit. While this seems negligible per transaction, it accumulates significantly over time, especially with high-value tokens or high transaction volumes.

The composer holds the full deposit amount before the LayerZero transfer, but after **IOFT.send** executes, the dust remainder cannot be retrieved. **MoreVaultsComposer** has functions for deposits, refunds, and cross-chain operations, but no administrative rescue function exists. Users lose fractional amounts on every cross-chain deposit, the protocol accumulates locked value it cannot access, and the **Deposited** event emits **_assetAmount** showing the full pre-truncation amount while the actual cross-chain transfer sends less, creating accounting discrepancies for off-chain indexers and dashboards.

Recommendation

Add a rescue function for accumulated dust.

3S-Vaults-M06

Missing overflow protection in assembly-based accounting functions

Id	3S-Vaults-M06
Classification	Medium
Impact	High
Likelihood	Low
Category	Bug
Status	Addressed in #ee1cfc7 .

Description

The `_accountFacets()` function performs arithmetic operations inside an assembly block without overflow protection, potentially allowing malicious or buggy facets to corrupt the protocol's accounting system.

```
// decode return values
let decodedAmount := mload(retOffset)
let isPositive := mload(add(retOffset, 0x20))
// if the amount is positive, add it to the total assets else add to debt
if isPositive { _totalAssets := add(_totalAssets, decodedAmount) }
if iszero(isPositive) { debt := add(debt, decodedAmount) }
```

In Solidity 0.8.x, arithmetic operations outside assembly blocks have automatic overflow protection. However, operations inside assembly blocks bypass these checks. If a compromised or buggy facet returns `type(uint256).max` or near-maximum values, the additions will silently wrap around to small numbers, corrupting `totalAssets`. This affects all vault operations: **deposit**, **withdraw**, **redeem**, **convertToShares**, and **convertToAssets**, potentially causing incorrect share calculations and value loss.

While current facets appear safe, the protocol's modular architecture allows adding new accounting facets. Without overflow protection, one malicious or buggy facet can compromise the entire system. For comparison, `_accountAvailableAssets` performs additions outside assembly with automatic protection.

Recommendation

Add explicit overflow checks in the assembly block.

3S-Vaults-M07

Refund function will always revert due to incorrect OFT address usage

Id	3S-Vaults-M07
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Addressed in #d50ba10 .

Description

The **refundDeposit** function attempts to call LayerZero's **send** on the underlying ERC20 token address instead of the OFT adapter address, causing all refund operations to revert and permanently locking user funds.

```
IERC20(deposit.tokenAddress).forceApprove(deposit.oftAddress, deposit.assetAmount);
IOFT(deposit.tokenAddress).send{value: deposit.msgValue}(
    refundSendParam, MessagingFee(deposit.msgValue, 0), deposit.refundAddress
);
IERC20(deposit.tokenAddress).forceApprove(deposit.oftAddress, 0);
```

The function uses **deposit.tokenAddress** (the underlying ERC20 like USDC) to call **send**, but standard ERC20 tokens don't implement LayerZero's **IOFT** interface. Only OFT adapter contracts have the **send** function. When refund is triggered, the call reverts and user funds remain locked in the composer with no recovery mechanism. This breaks the critical safety feature for handling failed cross-chain deposits.

Recommendation

Use **deposit.oftAddress** instead of **deposit.tokenAddress**:

```
IERC20(deposit.tokenAddress).forceApprove(deposit.oftAddress, deposit.assetAmount);
IOFT(deposit.oftAddress).send{value: deposit.msgValue}(
    refundSendParam, MessagingFee(deposit.msgValue, 0), deposit.refundAddress
);
```

```
IERC20(deposit.tokenAddress).forceApprove(deposit.oftAddress, 0);
```

3S-Vaults-M08

Missing slippage protection in vault deposit and withdrawal operations

Id	3S-Vaults-M08
Classification	Medium
Impact	Medium
Likelihood	Medium
Category	Bug
Status	Addressed in #119aa54 .

Description

The **deposit**, **mint**, **withdraw**, and **redeem** functions in **VaultFacet** lack slippage protection parameters, exposing users to unfavorable exchange rates when the vault's **totalAssets** changes unexpectedly between transaction submission and execution. In standard ERC4626 vaults, asset-to-share typically only increases through yield generation, making slippage protection less critical. However, this protocol's architecture introduces multiple sources of rapid and unpredictable changes to the asset-to-share exchange rate:

1. Debt fluctuations. The vault's **totalAssets** is calculated as assets minus debt. If debt increases between when a user previews their deposit/withdraw and when the transaction executes, they receive fewer shares (on deposit) or fewer assets (on withdraw) than expected
2. Multi-asset price volatility. The vault holds different types of assets with varying volatilities. Price movements in any held asset affect normalized **totalAssets** calculation, causing the exchange rate to fluctuate.
3. Cross-chain accounting updates. For cross-chain vaults, accounting information from spoke chains can update **totalAssets** asynchronously, causing sudden changes in the exchange rate.

Recommendation

Add slippage protection parameters to all four ERC4626 core functions.

3S-Vaults-M09

maxDeposit and maxMint functions don't account for whitelist user deposit limits

Id	3S-Vaults-M09
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Addressed in #56492fc .

Summary

The **maxDeposit** and **maxMint** view functions in VaultFacet only check global deposit capacity while ignoring user-specific whitelist limits. This can mislead both users and external contracts integrating with the protocol, potentially leading to failed transactions and integration issues.

Description

When whitelist mode is enabled in the vault, each whitelisted user has a specific deposit limit enforced during actual deposit/mint operations. But, the **maxDeposit** and **maxMint** functions do not consider these individual limits and only return values based on the vault's global deposit capacity.

This creates two problems:

1. Direct user impact: Users querying these functions see incorrect maximum deposit/mint amounts that exceed their actual whitelist limits, leading to failed transactions when they attempt to deposit the reported maximum.
2. External contract integration impact: Smart contracts that integrate with the vault mostly rely on these view functions to calculate how much to deposit will break.

Recommendation

1. Modify both functions to account for whitelist limits when enabled

2. Return the minimum between the global deposit capacity and the user's remaining whitelist allowance

3S-Vaults-M10

Lack of Finalization Check in finalizeRequest function

Id	3S-Vaults-M10
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Addressed in #351eec8 .

Summary

The **finalizeRequest** function in the BridgeFacet contract lacks a critical check to prevent the same request from being finalized multiple times. This allows for the re-execution of an already finalized request, as anyone can call the **finalizeRequest** function.

Description

The function correctly checks if a request has been fulfilled (meaning the cross-chain accounting is done) using the **ds.guidToCrossChainRequestInfo[guid].fulfilled** flag. But, the function never marks the request as finalized after executing it. So an attacker could repeatedly call this function with a valid **guid** that has already been processed. As long as the call is made within the one-hour timeout window, the function will execute the underlying action (e.g., deposit, mint, withdraw) again, leading to a duplication of the original request's outcome.

Recommendation

To fix this issue, add a new boolean flag, such as **finalized**, to the **MoreVaultsLib.CrossChainRequestInfo** struct to track the finalization status of a request. After the action has been successfully executed, set the **finalized** flag for the given **guid** to true and revert **finalizeRequest** call if the flag is already set to true.

3S-Vaults-M11

Deposit whitelist bypass via share transfers allows unlimited vault exposure

Id	3S-Vaults-M11
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Addressed in #d24fae6 .

Description

The **depositWhitelist** mechanism in **VaultFacet._validateCapacity** is designed to limit the maximum deposit amount per whitelisted address. However, the implementation checks the user's current share balance rather than tracking cumulative deposits:

```
uint256 userDepositedAssets =
    _convertToAssetsWithTotals(balanceOf(receiver), totalSupply(), newTotalAssets,
    Math.Rounding.Ceil);
if (ds.depositWhitelist[receiver] < userDepositedAssets + newAssets) {
    revert ERC4626ExceededMaxDeposit(...);
}
```

This can be trivially bypassed:

1. Alice has a whitelist limit of 1000 asset tokens
2. Alice deposits 1000 asset tokens → receives shares
3. Alice transfers all shares to Bob (or a secondary address she controls)
4. **balanceOf(Alice)** = 0, so the check passes
5. Alice deposits another 1000 asset tokens
6. Repeat steps 3-5 indefinitely

Alice can accumulate unlimited exposure to the vault by cycling her shares through different addresses, completely defeating the purpose of the whitelist restriction.

Recommendation

Track cumulative deposits per address rather than current balance.

3S-Vaults-M12

Single-timestamp accrual lets users skip performance fees within the same block

Id	3S-Vaults-M12
Classification	Medium
Impact	Medium
Likelihood	Medium
Category	Bug
Status	Addressed in #313996c .

Description

The **VaultFacet._accrueInterest** function, which charges the vault's performance fee on profits, records a timestamp and prevents charging the fee more than once for the same timestamp. Because of this, if within a single block an action calls **_accrueInterest**, then yield (profit) is realized, and finally a user performs a withdraw, that user will not pay the performance fee on that profit - unlike the remaining depositors. A malicious user can try to frontrun the profit realization to avoid paying the fee.

Recommendation

Allow fee accrual more than once per block when there is new positive growth.

3S-Vaults-M13

pauseFacet can be Dosed by a malicious linked vault

Id	3S-Vaults-M13
Classification	Medium
Impact	Medium
Likelihood	Medium
Category	Bug
Status	Addressed in #eed26a6 .

Description

In the **VaultsFactory** contract, anyone can register a vault and link it to a facet via **link**. The same contract exposes **pauseFacet**, which is intended to restrict a facet and pause all vaults linked to that facet. The issue is that a malicious user can register (link) a vault whose **pause()** reverts. When **pauseFacet** iterates and calls **IVaultFacet(vaults[i]).pause()**, a single revert will bubble up and revert the entire transaction, preventing legitimate vaults from being paused.

```
function pauseFacet(address _facet) external onlyOwner {
    address[] memory vaults = _linkedVaults[_facet].values();
    _setFacetRestricted(_facet, true);
    for (uint256 i = 0; i < vaults.length;) {
        IVaultFacet(vaults[i]).pause(); // ← a malicious vault can revert here
        unchecked { ++i; }
    }
}
```

Recommendation

A possible solution is try / catch

3S-Vaults-M14

Unlimited **forceApprove** to **vault** enables asset drain on upgrade/malicious vault

Id	3S-Vaults-M14
Classification	Medium
Impact	High
Likelihood	Low
Category	Bug
Status	Addressed in #06e8e1d .

Description

In **ERC4626Facet.genericAsyncActionExecution** the code grants an unlimited allowance to the external **vault** before making the call, and only resets it after the call returns:

```
IERC20(balances.asset).forceApprove(vault, type(uint256).max);  
(execution.success, execution.result) = vault.call(execution.fixedData);  
IERC20(balances.asset).forceApprove(vault, 0);
```

Even if the whitelisted vault is initially safe, an upgrade (or later compromise) can modify its implementation to invoke **transferFrom** during that call and siphon all **balances.asset**, leveraging the temporary max allowance.

Recommendation

Approve only the exact amount needed for the action, then reset to 0.

3S-Vaults-L01

Price per share inflation due to unminted performance fees

Id	3S-Vaults-L01
Classification	Low
Impact	Low
Likelihood	High
Category	Bug
Status	Acknowledged

Description

The current High Water Mark (HWM) implementation charges performance fees per-user only when that specific user performs an action (deposit, withdraw, mint, or redeem). This creates a price per share inflation issue because **totalSupply** does not include pending fee shares for inactive users.

When User A has unrealized profit with accrued but unminted fee shares, and User B performs a deposit or withdrawal, the protocol calculates price per share using **totalAssets** (which includes User A's profit) divided by **totalSupply** (which excludes User A's pending fee shares). This results in an artificially inflated price per share, giving User B a better rate than they should receive. User B effectively benefits from dilution that hasn't happened yet but should be reflected in the share price calculation.

Recommendation

There is no perfect solution for this issue but a possible one is to periodically calculate all accrued fees off-chain and update the protocol. The downside is that in case of an off-chain error, users have no protection.

3S-Vaults-L02

Inconsistent address usage in deposit whitelist capacity handling

Id	3S-Vaults-L02
Classification	Low
Impact	Low
Likelihood	High
Category	Bug
Status	Addressed in #fe69259 .

Summary

There is a logical inconsistency in how the deposit whitelist is handled in the VaultFacet.sol contract. The deposit capacity check is performed against the receiver's address, while the subsequent update to the deposit capacity is applied to the caller's (msg.sender) address.

Description

When a deposit is made where the caller and receiver are different, the contract's logic for handling the deposit whitelist is inconsistent.

The **_validateCapacity** function is called with the receiver's address. This function checks if ds.depositWhitelist[receiver] has sufficient capacity for the deposit amount.

```
function deposit(uint256 assets, address receiver)
    public
    virtual
    override(ERC4626Upgradable, IVaultFacet)
    whenNotPaused
    returns (uint256 shares)
{
    ...
    _validateCapacity(receiver, newTotalAssets, assets); // @audit
    ....
    _deposit(msgSender, receiver, assets, shares);
}
```


After the deposit is processed, the **_deposit** function calls **_changeDepositCap** with the caller's address (**msg.sender**). This function then reduces the deposit capacity for the caller.

```
function _deposit(address caller, address receiver, uint256 assets, uint256 shares)
internal virtual override {
    super._deposit(caller, receiver, assets, shares);
    MoreVaultsLib.MoreVaultsStorage storage ds = MoreVaultsLib.moreVaultsStorage();
    ds.lastTotalAssets = ds.lastTotalAssets + assets;
    _changeDepositCap(ds, caller, assets, true); // @audit
}
```

Recommendation

Ensure that deposit capacity validation and updates are performed consistently for the same address.

3S-Vaults-L03

Shared storage between facets prevents multicall execution and creates pause state conflicts

Id	3S-Vaults-L03
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Acknowledged

Description

MulticallFacet, **BridgeFacet**, and **VaultFacet** inherit OpenZeppelin's **ReentrancyGuard** and **PausableUpgradeable**, causing them to share the same storage slots for reentrancy locks and pause states.

Issue 1: Reentrancy guard conflict

MulticallFacet and **BridgeFacet** both inherit **ReentrancyGuard**, sharing the same **_status** storage slot. When **MulticallFacet.executeActions** with **nonReentrant** modifier is called, any attempt to execute **BridgeFacet** functions with **nonReentrant** (like **initVaultActionRequest** or **finalizeRequest**) will revert because both check the same flag.

Issue 2: Pause state conflict

VaultFacet and **BridgeFacet** both inherit **PausableUpgradeable**, sharing the same **_paused** storage slot. This creates an unintended coupling:

- When **VaultFacet.pause** is called, it also pauses **BridgeFacet**
- Both facets become paused/unpaused together, even though they should be independently controllable

Recommendation

Use separate storage namespaces for each facet's guards

3S-Vaults-L04

Missing access control check on
`setDefaultCrossChainAccountingManager` function

Id	3S-Vaults-L04
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Addressed in #d113178 .

Summary

The `setDefaultCrossChainAccountingManager` function lacks access control, allowing any address to modify the `defaultCrossChainAccountingManager` state variable.

Description

The function `setDefaultCrossChainAccountingManager` in the `VaultsRegistry` contract is missing the `onlyRole(DEFAULT_ADMIN_ROLE)` modifier that should restrict access to only admin. This means any external address could set the default cross-chain accounting manager.

However, this issue currently has no immediate impact because ,the `defaultCrossChainAccountingManager` variable is not used anywhere in the codebase.

Recommendation

If the `defaultCrossChainAccountingManager` variable is intended to be used in future upgrades, add the `onlyRole(DEFAULT_ADMIN_ROLE)` modifier to restrict access. Otherwise, remove both the state variable and the function entirely.